Data Science Program

Capstone Report - Spring 2026

Member Monitor Capstone Project

Morgan Fisher

A capstone presented in fulfillment of the requirements for the degree of Bachelor's of Science in Data Science

supervised by
Prof. Sushovan Majhi, Prof. Sarah Burnett

Preface

How do congressional members vote? What bills do they introduced? The bills they support? Member Monitor is a full stack web application that provides answers to these questions. Taking publicly available data from congressional websites, Member Monitor filters and cleans the data to display it on a publicly available website where users can seek answers to their questions.

Acknowledgements

I want to thank Professor Burnett for advising me on this project. I also want to thank my peers at GW for offering feedback on this project throughout the semester as well as my fellow students in DATS 4001.

This project is accumulation of the skills I have learned from various internships and projects I have been involed in throughout my time at GW. During my first internship with Aidan Health, I worked on my first long term project and learned how systems interact with one another. Interning at the Sunwater Institute, I contributed to their political database adding interactive visualizations, which inspired the setup and visualization style of this web application. Being involved with the GW Club Running team, I coded a new website for the team, helping me learn frontend development skills. Interning at the House of Representatives, I gained important insights on how Congress works and which data is valuable in a congressional setting. Lastly, this past summer working on a software engineering team at Dell, I learned how to integrate a backend and database to a frontend. I wanted to thank everyone involved in these learning experiences who helped me along my coding journey and motivated me to pursue this capstone project.

Abstract

Member Monitor addresses the challenge of accessing and interpreting congressional data, which is often fragmented, complex, and difficult for the public to navigate. This project presents a full stack web application that collects, processes, and organizes publicly available congressional data into a structured and accessible format. A custom data pipeline integrates API data and web-scraped sources, transforming raw data into a relational PostgreSQL database that supports efficient querying. The application's Flask backend and React frontend enable users to explore member voting records, bill sponsorships, and legislative trends through an intuitive interface and interactive visualizations. The application demonstrates that a well-designed data pipeline combined with user-focused design can improve transparency and usability of political data, providing timely insights into congressional activity.

Keywords

data pipeline; web application; congressional data; data visualization; PostgreSQL; Flask; React; D3.js; political transparency; full stack development

Contents

1 Introduction **4**

2 Related Work **4**

3 Solution **4**

4 Results and Discussion **6**

4.1 Pipeline 6

4.2 Database 7

4.3 Backend 8

4.4 Frontend 8

4.5 Visualizations 9

5 Discussion **12**

6 Conclusion **12**

1 Introduction

The accessibility and transparency of political information exist as challenges for the general public. Although official sources such as congress.gov[1] publish detailed legislative and voting data, these platforms are often difficult to navigate and lack intuitive tools for quickly extracting meaningful insights. Existing third-party legislative databases attempt to address this gap, but many suffer from limited usability or require paid access, creating additional barriers for users seeking to understand the actions of their representatives.

Member Monitor addresses these limitations by providing a fast, user-friendly interface for exploring congressional voting behavior. The application uses data science techniques to collect, process, and organize publicly available government data. The platform enables users to search for their representatives and review their voting history from the most recent congressional session, promoting greater transparency and civic engagement.

2 Related Work

Many other political databases exist. Some of these databases are behind pay walls and others are difficult to navigate. Others work just fine. This project is less based on already existing works but rather to create a new and original, user-friendly, cherry blossom-themed political database so users can inquire about members of congress.

3 Solution

Member Monitor is a full stack web application with a data pipeline, database, backend, and frontend. All these elements are integrated with one another through API connections and servers. They work together to run the website, which is publicly available at `https://www.mf-member-monitor.com/`.

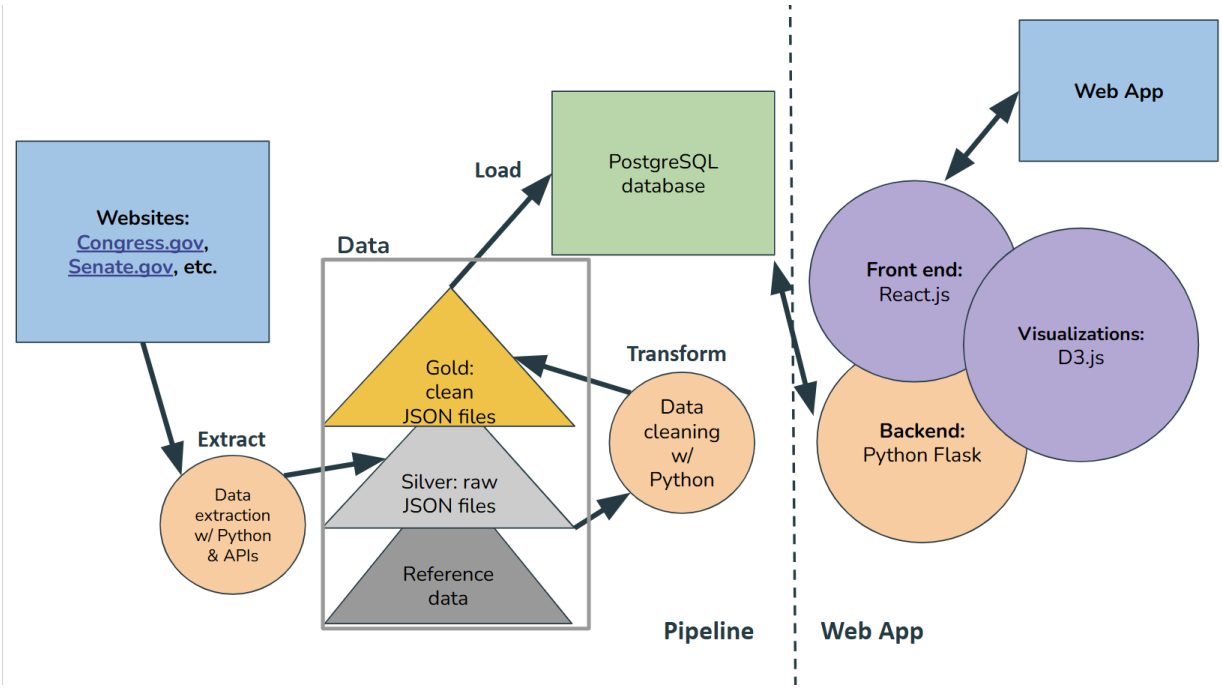

Figure 1: Architecture of Member Monitor

Figure 1 breaks down each component of Member Monitor. The architecture can be broken down into two main parts, the pipeline and the web application. First in the pipeline, data is extracted from congressional websites, such as congress.gov[1], senate.gov[2], and house.gov[3]. The data is extracted using python code which makes both API calls to congress.gov and also performs web scraping on public XML files for data unavailable via API. After extracting the data, it is landed as JSON files in a folder named silver. Second, is the transformation process. Using python, the silver data JSON files are cleaned by extracting specific variables and formatting them in a simple JSON format. These JSON files are landed in the gold folder, meaning they are ready to be sent to the PostgreSQL database. Third is the loading of the data. The python connects to the database which is hosted on a server. All data from the gold folder is upserted into the database.

The second main part is the web application consisting of the backend and frontend. The backend is hosted on the server, in addition to the database. Using python Flask, the backend connects to the database and extracts data based on queries triggered by the frontend. The frontend is built using React.js. When data needs to be loaded, the frontend triggers the backend which sends API calls indicating which data needs to be extracted from the database and sent back to the frontend. Within the frontend are several interactive visualizations which are built using D3.js. These visualizations query data to then be displayed within interactive charts that the user can modify based on the user-side filtering.

4 Results and Discussion

Considering the project's architecture, let us further breakdown each component and how it functions.

4.1 Pipeline

Referring back to Figure 1, the pipeline is a group of coding scripts that perform the extract, transform, and load steps in one run. The main script run_pipeline.py runs the extract, transform, and load elements in around 1-5 minutes, depending on the amount of new data to be extracted.

The extract scripts are separated into three categories: API, XML, and Build. The API script issues requests to retrieve data from the congress.gov to API. For member the data, the API will extract all data available at the time the call was made. Some of the data that is bulkier, such as the bills data, will only make calls for new data that has been added since the last time the pipeline has been run. For example, to gather bill sponsorships and policy areas, an API call must be made for each bill. Because of the number of bills, this would take a very long time (over an hour). So, the code looks at the existing storage of data and builds off of it, only extracting the latest data. The XML script extracts Senate roll call data and House/Senate voting data. This data is not available via APIs, so the data is extracting using web scraping. The XML data is scraped via a URL and converted to JSON format. Similar to the APIs, only the newest data is being scraped when the code runs as it looks at the existing data and takes the newest votes since. Lastly, the build script "builds" the Senate vote party totals based on the existing votes and member data that had already been extracted.

The transform scripts take the raw extracted data and "transform" them into clean JSON files with specific variables. A transform script exists for each table (discussed later in the database section) and some scripts have simple transformations while others require additional cleaning and changing of variables. For example, the members script requires both changing the member_id for Senators because the initial ID was inconsistent with the bioguide ID's used for the House. Additionally, state names needed to be transformed into their abbreviated version using reference data (a file that indicated all state names and their abbreviations). Additionally, each script has checks to make sure that the data and it's unique ID are in fact unique. If there exists repeated data, there will be an error that is raised. Throughout the project, data has arisen that has broken the pipeline. Using python there has been increased safeguards that have been implemented to ensure repeat data is being handled correctly in the case it arises. Lastly, once each script has cleaned data in JSON format, they are landed in the gold folder.

The load script is a single script that establishes a connection to the database, then takes each table (represented by their own JSON file) and upserts it into the database. Within the load script, the database is established with safeguards connected. If missing data exists, such as a bill_id in the votes table not existing in the bills table, the script will not load that votes record into the database until the bill_id

exists.

4.2 Database

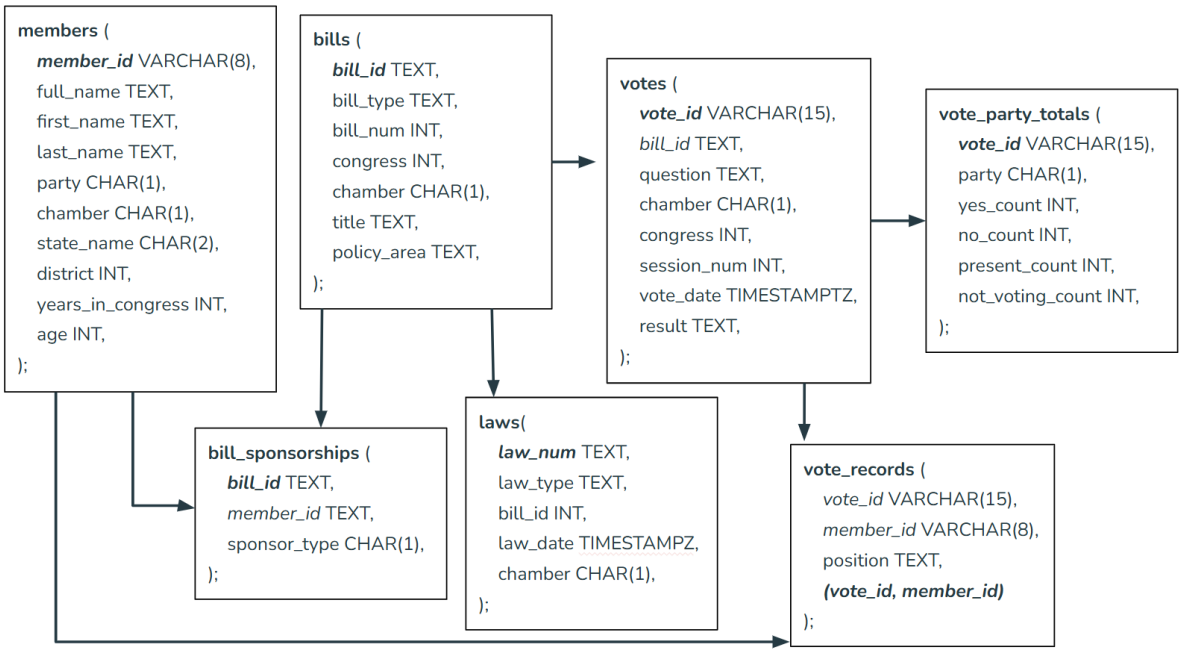

Figure 2: Database Relationships

The database relationship diagram shown in Figure 2 presents an overview of the database structure. As mentioned, each table exists as a JSON file that is upserted into the database from the pipeline. These tables connect to each other to allow for querying on the backend.

The two core tables that connect to all others are the members and the bills table. The members table contains one unique row (or record) for each member, providing unique information about that member (name, party, state, district, age, years in congress). The bills table contains one unique row for each bill. Additionally, it contains details about that bill, specifically the policy area which becomes an important data point for filtering on the frontend.

The laws table is an extension of the bills table. Laws contain unique rows for each law. This table is small as not many laws are created compared to the number of bills. The law table connects to the bills table by the bill that law originated as.

The bill_sponsorships table and merges the members and bills table as it has a unique row for each member/bill combination. So each time a member sponsors or cosponsors a bill, that forms a unique row in the sponsorships table.

The votes table has a unique row for each vote that occurred. Key features of the votes table include the bill the vote was on, the date of the vote, and the outcome. The vote_party_totals table is an extension of the votes table, containing a unique row for every vote/party combo. So for each vote their

exists a Democrat row and a Republican row which each include the breakdown of the votes among the parties.

Lastly, the vote_records table provides the main element of this project, a unique row for each member/vote combination showing how that member voted. This results in 435 rows for each vote in the house and 100 rows for each vote in the senate. Through this table structure, the web application can quickly query the database for specific data to provide a fast acting application.

4.3 Backend

The backend is built using pythonFflask and consists of two main parts: the connection to the database and the structuring of the API calls. First, the backend connects to the database and creates models of each table that allow for simple querying.

The larger part of the backend is the API routes. The first route "/api/members" gathers all the members so that they may be displayed in the member search dropdown on the home page. The API routes "/api/member/<string:member_id>" and "/api/member/<string:member_id>/" sponsorships' take a member_id input from the member search and gathers that member's data from the members table and all data of that member in the vote_records and bills_sponsorships tables and displays it on the generated specific member page. Lastly, the visualization API routes gather all the necessary data to build the interactive visualization. There is a unique API call for each visualization as they each utilize different tables and connections.

4.4 Frontend

The frontend is built using React.js. React was chosen for this project as many components of the page are reused. Using React allowed for less code to be structured and a consistent look throughout the website. The website contains three main pages. First, the home page features the member search, the data visualization gallery, and the about section. Second, the member pages are unique to each member but all contain the same structure displaying votes, sponsored bills, and cosponsored bills. Third, the data visualization pages display the D3.js visualizations which contain many shared components such as titles, filters, and dimensions. In addition to the three main pages, their is a navigation menu at the top that can navigate the users to the other pages of the website.

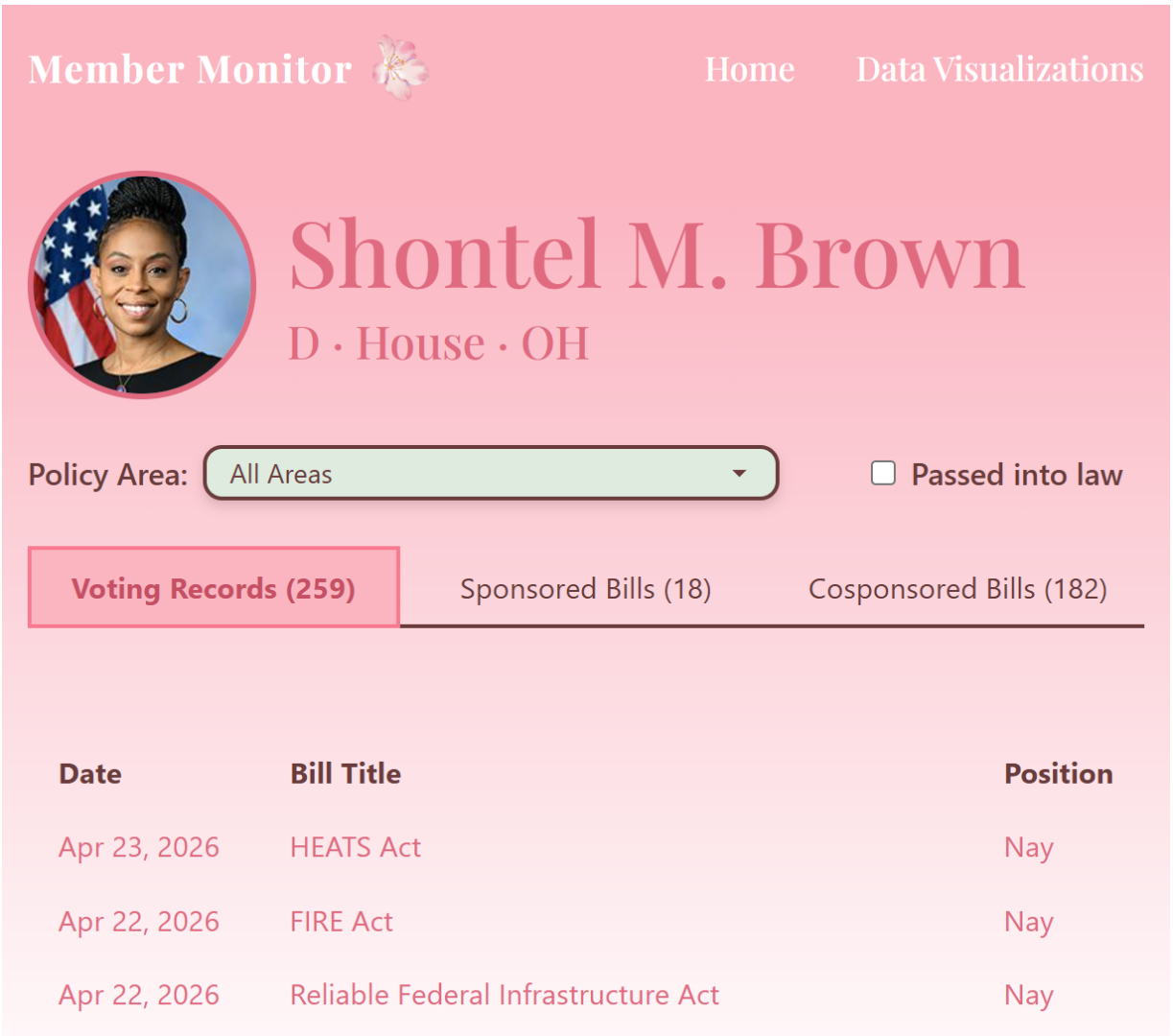

Figure 3: Screenshot of Member Page - Shontel Brown

Figure 3 provides a look at a unique member page. At the top, the member's photo and information is displayed. Below that are filters, so the users can filter the votes/bills by a specific policy area or they can filter by votes/bills on legislation that has been passed into law. Below the filters are the three tabs that provide the desired infromation. First the vote records displays the data, bill, and the member's position. The sponsored bills and cosponsored bills tabs provide a list of bills that member has sponsored or cosponsored. Each bill title acts as a link to that actual legislation on congress.gov[1].

4.5 Visualizations

The interactive visualization components of the website allow for users to explore the data presented. The website contains four interactive visualizations.

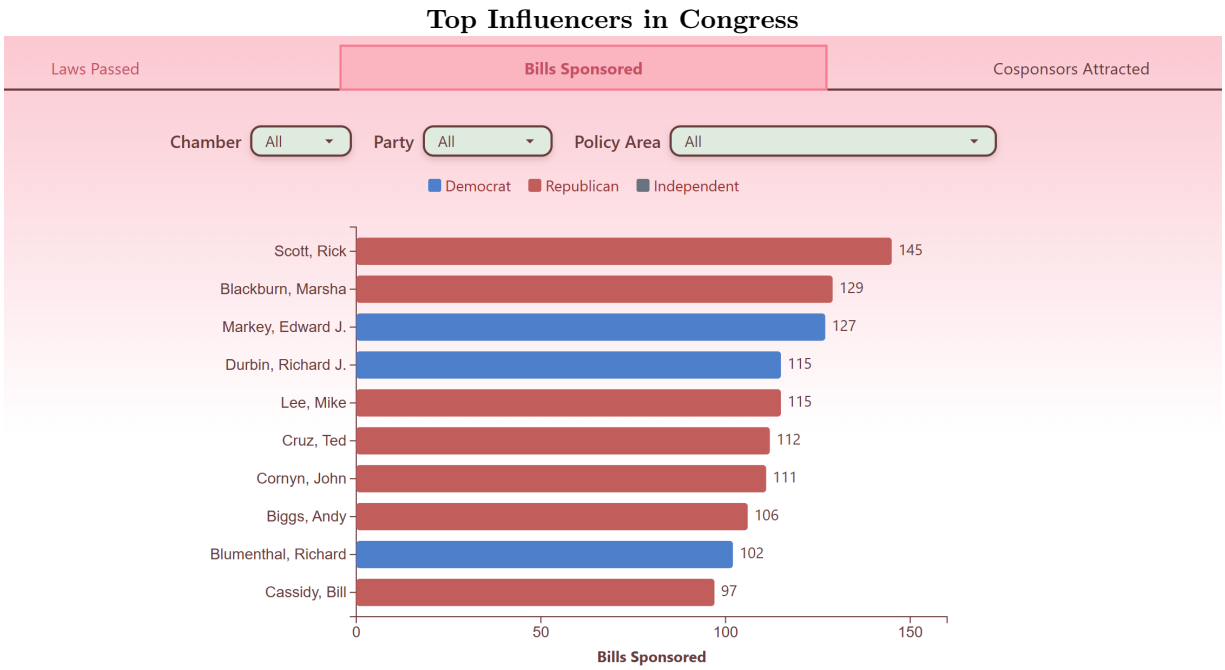

Figure 4: Interactive Visualization of the Top Influencers in Congress

The visualization displayed in Figure 4 takes sponsorship data to display a ranking of which members have the most influence in Congress. There are three tabs which users can browse through showing the top members for the number of laws passed, the number of bills sponsored, and the number of cosponsors attracted. Users may also filter by chamber, party, or policy area to explore the data. Additionally, each bar acts as a link to that member's own voting data page. This chart shows which members are the most active in Congress getting legislation sponsored or passed and gaining support from other members through attracting cosponsors.

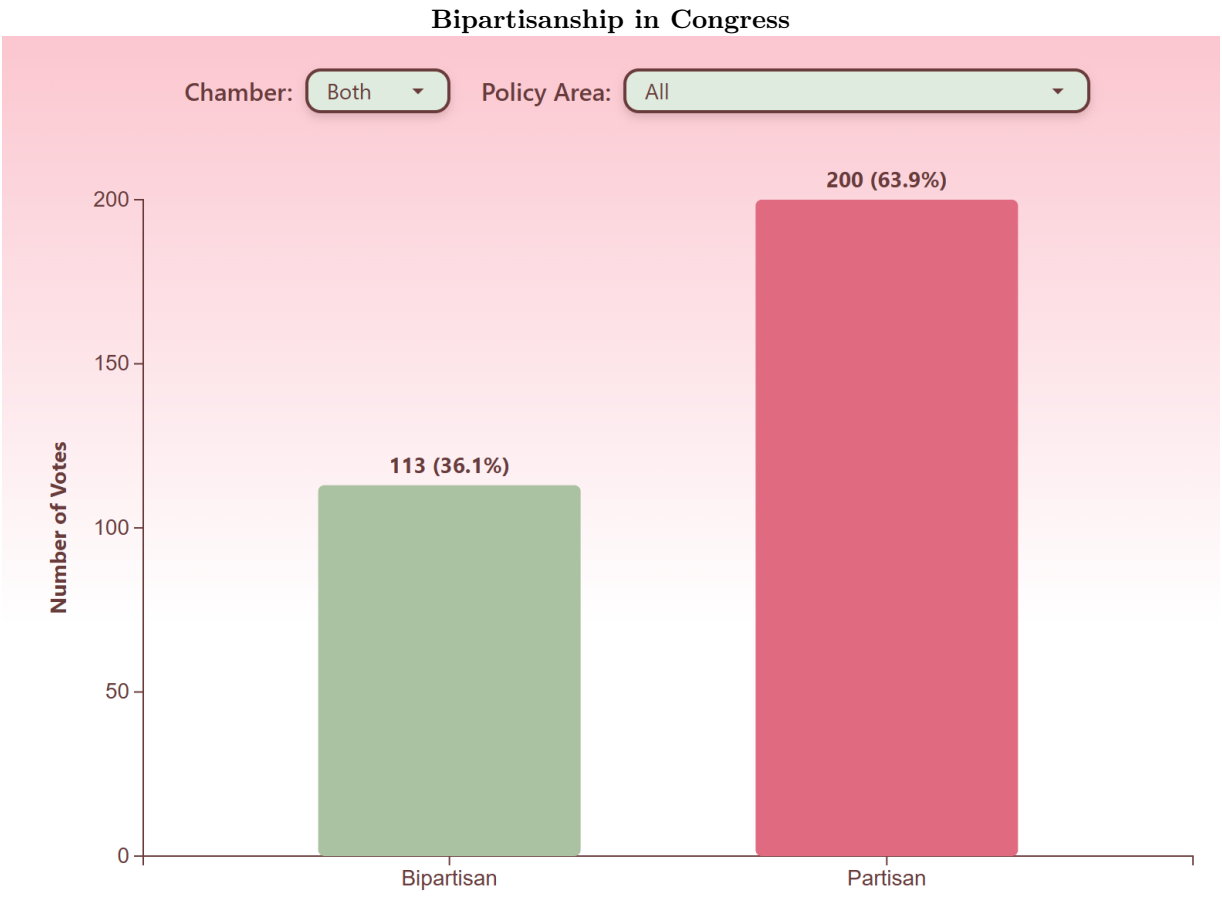

Figure 5: Interactive Visualization of the Bipartisanship in Congress

This second visualization displayed in Figure 5 uses the vote_party_totals data to display how often the two parties are agreeing and when are they divisive. The chamber and policy area filters on this visualization communicate key differences between the house and the senate in addition to party divides on various topics. Overall, this chart communicates that the two parties typically divide on their stance, but there are some areas that the parties agree.

The third visualization (not pictured) titled "How a Bill Survives Congress" displays the progression of how many bills are introduced to how many are voted on and how many become law. This chart relies on bill data and voting data. The interactive component allows users to filter by policy area so they can view how the data may change looking at specific bill topics. Overall, this visualization communicates how many bills are proposed in Congress but very few are voted on and even fewer are passed into law.

The last visualization (not pictured) titled "Congressional Activity Over Time" utilizes the "vote date" variable to display how votes are distributed among a Congressional session. The chart contains three filters: Chamber (House/Senate), Time Period (Day/Week/Month), and Policy Area. This chart shows that certain time periods have increased activity, and using the filters you can see that some topics have time periods with increased attention.

5 Discussion

This project uses many different tools, which have each provided their own challenges in learning and integrating with the rest of the project. These tools include: Python (Flask), APIs, web scraping, pieplenes, PostgreSQL, React.js, D3.js, Docker, and hosting (Both frontend and server hosting). In addition to learning these tools, figuring out which tools to use and how they work together posed many challenges. Yet these challenges were overcome and no significant challenges remain.

Using all these tools together have resulted in an open-source, entirely code-based application. This differs from other congressional databases that are not open-source and often require a paid subscription. This project is also unique in its look and aesthetic. The website is cherry blossom themed influencing its color scheme, layout, and features. Because this project is entirely code-based, it has allowed for complete customization which has been rewarding to explore.

6 Conclusion

Member Monitor meets its objective of improving understanding of congressional data through a fully integrated data science application. The project features:

- A functioning end-to-end data pipeline that extracts, transforms, and loads congressional data efficiently, with runtime typically under five minutes for incremental updates.
- A normalized relational database that enables fast and reliable querying across members, bills, votes, and sponsorship relationships.
- A backend API that supports data retrieval for both individual member analysis and aggregate visualizations.
- A responsive frontend interface that allows users to search for members, filter legislative activity, and explore interactive visualizations.
- A set of visual tools that highlight patterns in congressional behavior, including influence, bipartisanship, legislative success rates, and activity over time.

These results show that combining data engineering, backend development, and frontend visualization can produce a practical tool for increasing political transparency. The application provides a centralized and user-friendly alternative to existing platforms, lowering barriers for users seeking to understand congressional activity.

For the future, there are several opportunities for improvement. The dataset could be expanded to include past congressional sessions, additional data can be added (ex. committee data), and frontend can be expanded to ave more visualizations and have more information per member profile.

Overall, Member Monitor demonstrates how data science and software engineering can be applied to civic data to create accessible and informative public tools.

Website is available online via: `https://www.mf-member-monitor.com/`

References

[1] U.S. Congress, "Congress.gov," http://www.congress.gov, 2026, accessed: May 1, 2026.

[2] U.S. Senate, "Senate.gov," http://www.senate.gov, 2026, accessed: May 1, 2026.

[3] U.S. House of Representatives, "House.gov," http://www.house.gov, 2026, accessed: May 1, 2026.